

4444SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	VIMALA K	Reg. No.:	192424276
Section / Batch:	B.Tech(AI&DS)	Date:	22/8/2026

Code of Conduct

I VIMALA K 192424276 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:



Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A conversational AI system must identify grammatical relationships in user queries containing long-distance dependencies. The developers are comparing constituency parsing and dependency parsing.

Analyze how the two representations differ in capturing hierarchical and word-to-word relationships, and justify which approach is more suitable for extracting meaningful relationships from conversational text.

```
print("=" * 60)

print("1. CONSTITUENCY PARSING VS DEPENDENCY PARSING")

print("=" * 60)


print("\nCONSTITUENCY PARSING")

print("Represents a sentence as nested phrases.")

print("It identifies structures such as NP, VP and PP.")

print("It captures hierarchical relationships between phrases.")


print("\nExample:")

print("The student reads a book.")


print("Constituency Structure:")

print("S")

print("|")

print("++ NP -> The student")

print("|")

print("++ VP -> reads a book")

print("  |")

print("  ++ V -> reads")

print("  ++ NP -> a book")
```

```
print("\nDEPENDENCY PARSING")
```

```
print("Represents direct word-to-word relationships.")
```

```
print("\nDependency Structure:")
```

```
print("reads -> student")
```

```
print("reads -> book")
```

```
print("student -> The")
```

```
print("book -> a")
```

```
print("\nCOMPARISON")
```

```
print("Constituency parsing focuses on phrase hierarchy.")
```

```
print("Dependency parsing focuses on word-to-word relationships.")
```

```
print("\nCONCLUSION")
```

```
print("Dependency parsing is more suitable for conversational text")
```

```
print("when extracting meaningful relationships between words.")
```

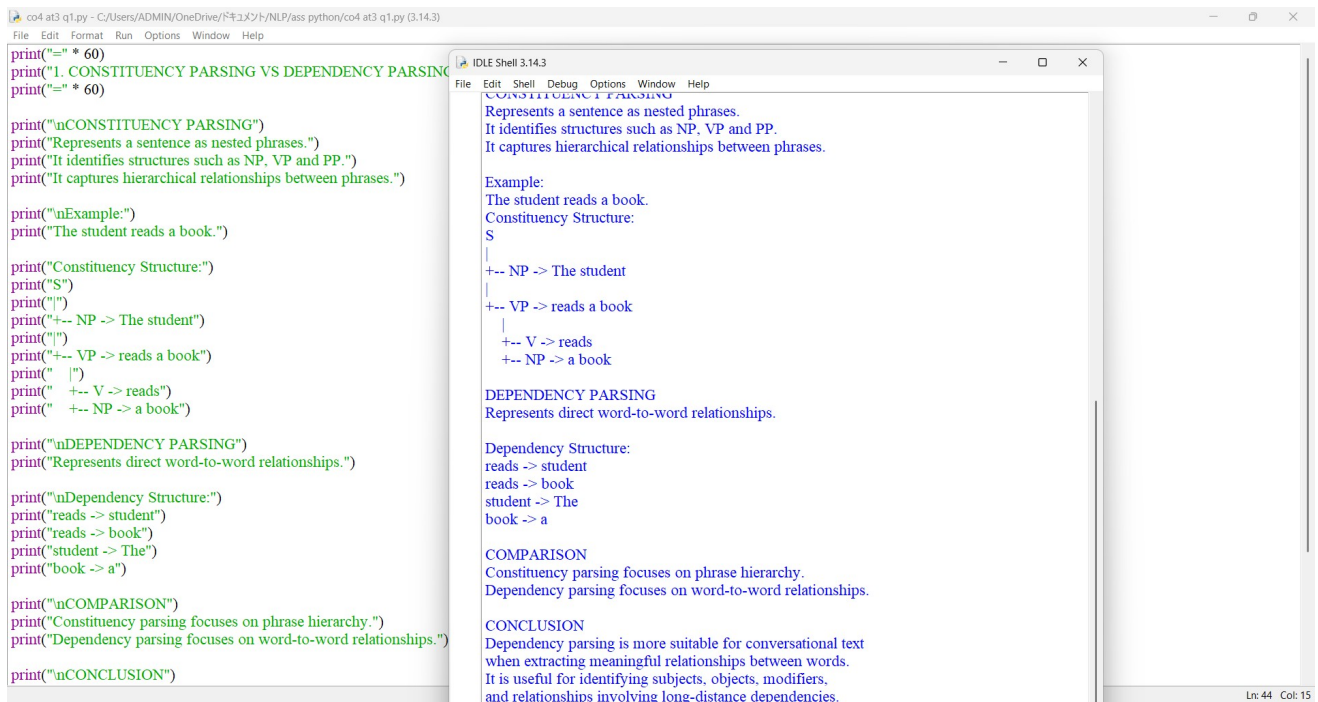
```
print("It is useful for identifying subjects, objects, modifiers,")
```

```
print("and relationships involving long-distance dependencies.")
```

```
print("=" * 60)
```

```
print("ANALYSIS COMPLETED")
```

```
print("=" * 60)
```



2. An NLP system processes partially typed search queries such as “best hotels near...” and “book a flight from Chennai to...”. The system currently uses conventional top-down parsing, while the developers are considering Earley parsing

Analyze how the two approaches behave when input is incomplete or ambiguous, and justify which parsing strategy is more appropriate for interactive NLP applications.

```

print("=" * 60)

print("2. TOP-DOWN PARSING VS EARLEY PARSING")

print("=" * 60)

queries = [

    "best hotels near...",

    "book a flight from Chennai to..."

]

print("\nINCOMPLETE INPUT")

```

for query in queries:

```
print("Query:", query)
```

```
print("\nTOP-DOWN PARSING")
```

```
print("Starts from the start symbol and predicts grammar rules.")
```

```
print("It may require backtracking when the prediction is wrong.")
```

```
print("Incomplete input can make prediction difficult.")
```

```
print("Ambiguous queries may require several alternative parses.")
```

```
print("\nEARLEY PARSING")
```

```
print("Can process incomplete input more effectively.")
```

```
print("Maintains possible parsing states.")
```

```
print("Handles ambiguous grammar.")
```

```
print("Supports left-recursive grammar.")
```

```
print("Can identify partial parse structures.")
```

```
print("\nEXAMPLE")
```

```
print("Input: book a flight from Chennai to...")
```

```
print("\nTop-Down Result:")
```

```
print("The parser may wait for the missing destination.")
```

```
print("\nEarley Result:")
```

```
print("The parser can maintain a partial structure:")
```

```
print("Action = Book")
```

```
print("Object = Flight")
```

```
print("Source = Chennai")
```

```
print("Destination = Incomplete")
```

```
print("\nCOMPARISON")
```

```
print("Top-Down Parsing:")
```

```
print("Ambiguity -> Requires backtracking")
```

```
print("Incomplete Input -> Less suitable")
```

```
print("Interactive NLP -> Limited")
```

```
print("\nEarley Parsing:")
```

```
print("Ambiguity -> Handles multiple parses")
```

```
print("Incomplete Input -> Handles partial input")
```

```
print("Interactive NLP -> More suitable")
```

```
print("\nCONCLUSION")
```

```
print("Earley Parsing is more appropriate for interactive NLP")
```

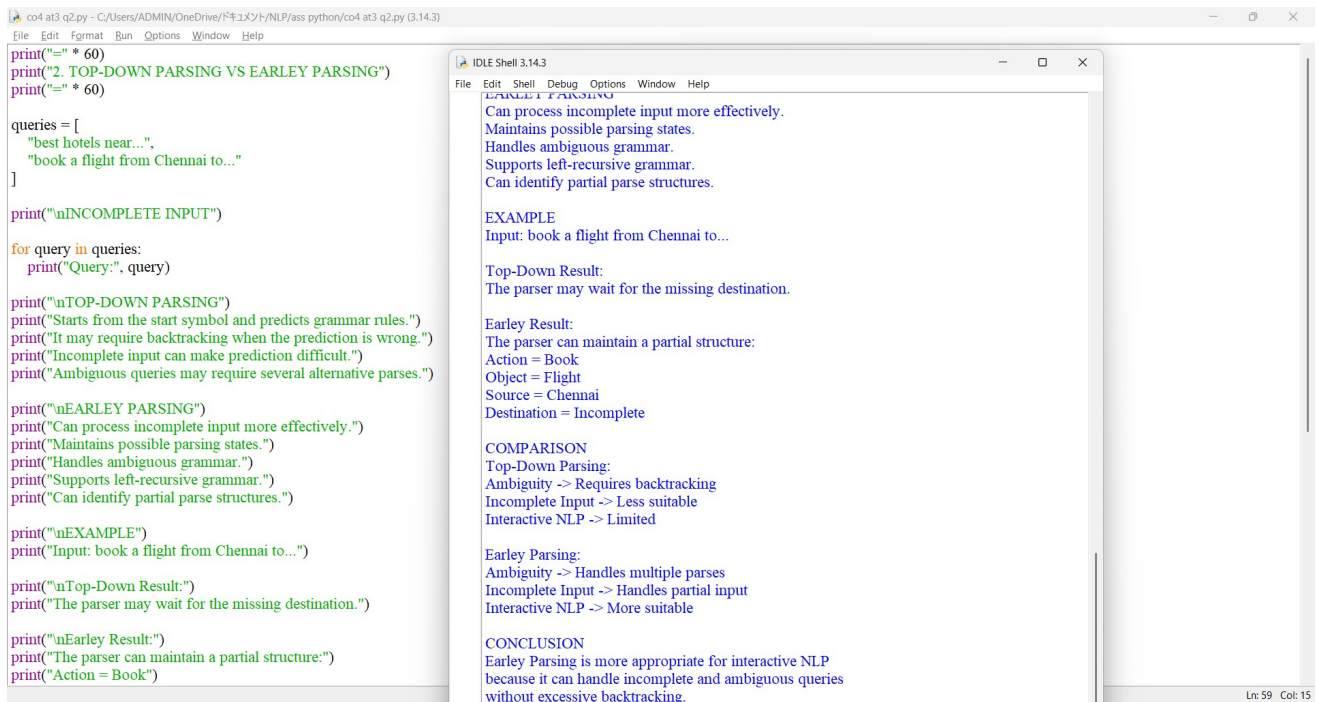
```
print("because it can handle incomplete and ambiguous queries")
```

```
print("without excessive backtracking.")
```

```
print("=" * 60)
```

```
print("ANALYSIS COMPLETED")
```

```
print("=" * 60)
```



3. A grammar-checking system must distinguish between multiple interpretations of structurally ambiguous sentences. The developers are comparing CFG, PCFG, and neural constituency parsers.

Analyze how each approach represents and resolves syntactic ambiguity, and justify which approach would provide the best balance between interpretability and practical accuracy.

```

print("=" * 60)

print("3. CFG VS PCFG VS NEURAL CONSTITUENCY PARSERS")

print("=" * 60)

sentence = "I saw the man with a telescope."

print("\nAMBIGUOUS SENTENCE")

print(sentence)

print("\nINTERPRETATION 1")

print("I used a telescope to see the man.")

```

```
print("\nINTERPRETATION 2")
```

```
print("The man had a telescope.")
```

```
print("\nCFG")
```

```
print("CFG represents possible grammatical structures.")
```

```
print("It can generate multiple parse trees.")
```

```
print("It does not assign probabilities to the alternatives.")
```

```
print("\nPCFG")
```

```
print("PCFG assigns probabilities to grammar rules.")
```

```
print("It ranks possible parse trees.")
```

```
print("The highest probability parse can be selected.")
```

```
print("\nNEURAL CONSTITUENCY PARSER")
```

```
print("Uses machine learning to learn syntactic patterns.")
```

```
print("Uses contextual information from the sentence.")
```

```
print("Can achieve high practical parsing accuracy.")
```

```
print("Can handle complex and ambiguous language effectively.")
```

```
print("\nCOMPARISON")
```

```
print("\nCFG:")
```

```
print("Interpretability = High")
```

```
print("Ambiguity Resolution = Limited")
```

```
print("Accuracy = Moderate")
```

```
print("Learning from Data = No")
```

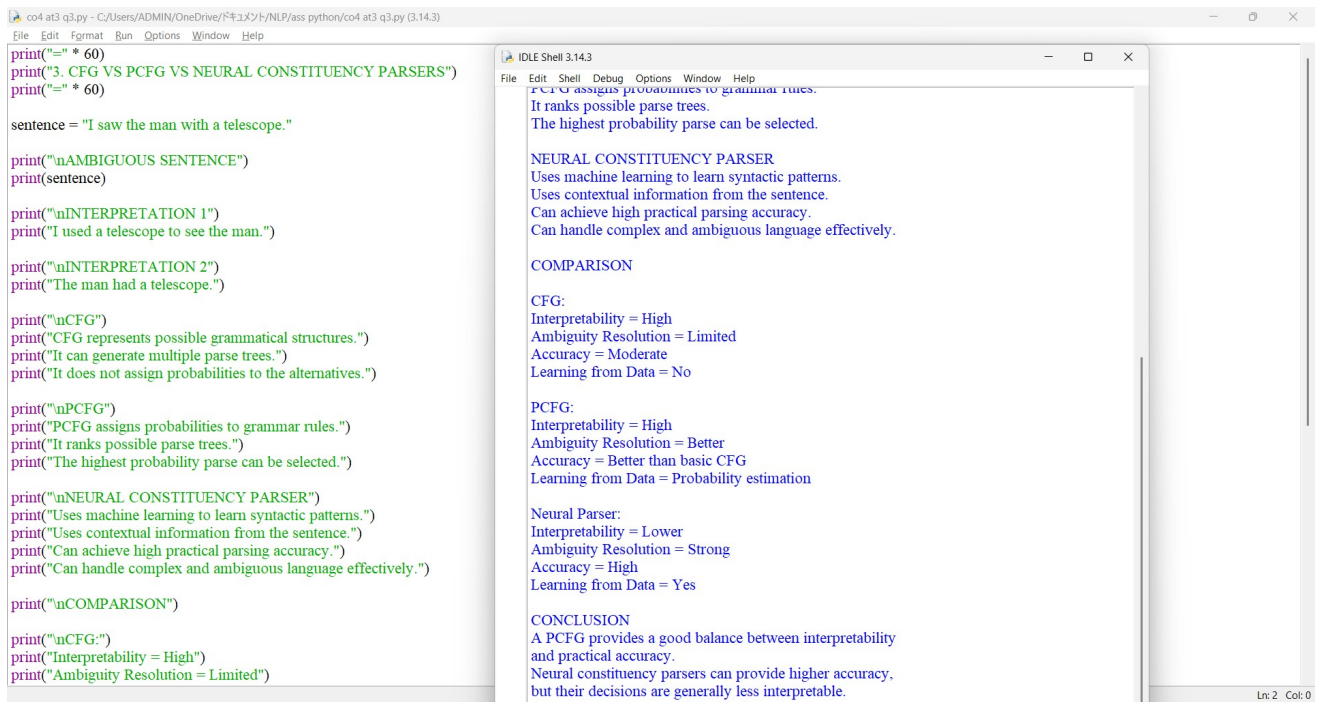


```
print("\nPCFG:")  
  
print("Interpretability = High")  
  
print("Ambiguity Resolution = Better")  
  
print("Accuracy = Better than basic CFG")  
  
print("Learning from Data = Probability estimation")
```

```
print("\nNeural Parser:")  
  
print("Interpretability = Lower")  
  
print("Ambiguity Resolution = Strong")  
  
print("Accuracy = High")  
  
print("Learning from Data = Yes")
```

```
print("\nCONCLUSION")  
  
print("A PCFG provides a good balance between interpretability")  
  
print("and practical accuracy.")  
  
print("Neural constituency parsers can provide higher accuracy,")  
  
print("but their decisions are generally less interpretable.")
```

```
print("=" * 60)  
  
print("ANALYSIS COMPLETED")  
  
print("=" * 60)
```



The screenshot shows a Python script in a text editor and its output in the IDLE Shell. The script compares three parsing methods: CFG, PCFG, and Neural Constituency Parsing. It uses the sentence "I saw the man with a telescope." as an example. The output in the shell shows the results of each method, including their interpretability, ambiguity resolution, accuracy, and learning from data capabilities.

```
co4 at3 q3.py - C:/Users/ADMIN/OneDrive/デスクトップ/NLP/ass python/co4 at3 q3.py (3.14.3)
File Edit Format Run Options Window Help

print("=" * 60)
print("3. CFG VS PCFG VS NEURAL CONSTITUENCY PARSERS")
print("=" * 60)

sentence = "I saw the man with a telescope."

print("\nAMBIGUOUS SENTENCE")
print(sentence)

print("\nINTERPRETATION 1")
print("I used a telescope to see the man.")

print("\nINTERPRETATION 2")
print("The man had a telescope.")

print("\nCFG")
print("CFG represents possible grammatical structures.")
print("It can generate multiple parse trees.")
print("It does not assign probabilities to the alternatives.")

print("\nPCFG")
print("PCFG assigns probabilities to grammar rules.")
print("It ranks possible parse trees.")
print("The highest probability parse can be selected.")

print("\nNEURAL CONSTITUENCY PARSER")
print("Uses machine learning to learn syntactic patterns.")
print("Uses contextual information from the sentence.")
print("Can achieve high practical parsing accuracy.")
print("Can handle complex and ambiguous language effectively.")

print("\nCOMPARISON")

print("\nCFG:")
print("Interpretability = High")
print("Ambiguity Resolution = Limited")
print("Accuracy = Moderate")
print("Learning from Data = No")

print("\nPCFG:")
print("Interpretability = High")
print("Ambiguity Resolution = Better")
print("Accuracy = Better than basic CFG")
print("Learning from Data = Probability estimation")

print("\nNeural Parser:")
print("Interpretability = Lower")
print("Ambiguity Resolution = Strong")
print("Accuracy = High")
print("Learning from Data = Yes")

print("\nCONCLUSION")
print("A PCFG provides a good balance between interpretability")
print("and practical accuracy.")
print("Neural constituency parsers can provide higher accuracy," data-bbox="418 361 648 376")
print("but their decisions are generally less interpretable.")
```

4. A multilingual NLP system must process sentences in which grammatical information depends on number, gender, tense, and person. The developers are considering feature structures and traditional context-free grammar rules.

Analyze how feature structures extend basic grammar representations and justify their usefulness for enforcing grammatical constraints in multilingual systems.

```
print("=" * 60)
```

```
print("4. FEATURE STRUCTURES VS CONTEXT-FREE GRAMMAR")
```

```
print("=" * 60)
```

```
print("\nBASIC CONTEXT-FREE GRAMMAR")
```

```
print("S -> NP VP")
```

```
print("NP -> Det N")
```

```
print("VP -> V NP")
```

```
print("\nPROBLEM")
```

```
print("Basic CFG does not directly represent:")
```

```
print("Number")
```

```
print("Gender")
```

```
print("Person")
```

```
print("Tense")
```

```
print("Agreement")
```

```
print("\nFEATURE STRUCTURES")
```

```
noun_features = {  
    "category": "Noun",  
    "number": "singular",  
    "gender": "neutral",  
    "person": "third"  
}
```

```
verb_features = {  
    "category": "Verb",  
    "tense": "present",  
    "number": "singular",  
    "person": "third"  
}
```

```
print("\nNoun Features:")
```

```
for key, value in noun_features.items():
```

```
    print(key, "=", value)
```

```
print("\nVerb Features:")

for key, value in verb_features.items():

    print(key, "=", value)


print("\nAGREEMENT CHECK")


if (noun_features["number"] == verb_features["number"] and
    noun_features["person"] == verb_features["person"]):

    print("Agreement = Correct")

else:

    print("Agreement = Incorrect")


print("\nFEATURE STRUCTURE ADVANTAGES")

print("1. Represents grammatical information.")

print("2. Enforces subject-verb agreement.")

print("3. Handles number differences.")

print("4. Handles gender information.")

print("5. Represents person.")

print("6. Represents tense.")

print("7. Useful for multilingual grammar.")


print("\nMULTILINGUAL APPLICATION")

print("English -> Number and person")

print("Hindi -> Number, gender and person")

print("Spanish -> Number, gender and person")

print("French -> Number, gender and agreement")
```

```

print("\nCONCLUSION")

print("Feature structures extend CFG by adding grammatical features.")

print("They are useful for enforcing grammatical constraints")

print("and improving multilingual NLP systems.")


print("=" * 60)

print("ANALYSIS COMPLETED")

print("=" * 60)

```

```

co4 at3 q4.py - C:/Users/ADMIN/OneDrive/デスクトップ/NLP/ass python/co4 at3 q4.py (3.14.3)
File Edit Format Run Options Window Help

print("=" * 60)
print("4. FEATURE STRUCTURES VS CONTEXT-FREE GRAMMAR")
print("=" * 60)

print("\nBASIC CONTEXT-FREE GRAMMAR")

print("S -> NP VP")
print("NP -> Det N")
print("VP -> V NP")

print("\nPROBLEM")
print("Basic CFG does not directly represent:")
print("Number")
print("Gender")
print("Person")
print("Tense")
print("Agreement")

print("\nFEATURE STRUCTURES")

noun_features = {
    "category": "Noun",
    "number": "singular",
    "gender": "neutral",
    "person": "third"
}

verb_features = {
    "category": "Verb",
    "tense": "present",
    "number": "singular",
    "person": "third"
}

print("\nNoun Features:")
for key, value in noun_features.items():

Noun Features:
category = Noun
number = singular
gender = neutral
person = third

Verb Features:
category = Verb
tense = present
number = singular
person = third

AGREEMENT CHECK
Agreement = Correct

FEATURE STRUCTURE ADVANTAGES
1. Represents grammatical information.
2. Enforces subject-verb agreement.
3. Handles number differences.
4. Handles gender information.
5. Represents person.
6. Represents tense.
7. Useful for multilingual grammar.

MULTILINGUAL APPLICATION
English -> Number and person
Hindi -> Number, gender and person
Spanish -> Number, gender and person
French -> Number, gender and agreement

CONCLUSION
Feature structures extend CFG by adding grammatical features.
They are useful for enforcing grammatical constraints
and improving multilingual NLP systems.

Ln: 4 Col: 0

```

5. A voice assistant must interpret commands containing verbs with different argument requirements, such as “give the book to John,” “sleep,” and “put the file on the desk.” The developers are considering subcategorization frames.

Analyze how subcategorization information helps determine valid sentence structures and justify its importance in semantic and syntactic analysis.

```

print("=" * 60)

print("5. SUBCATEGORIZATION FRAMES")

print("=" * 60)

```

```
verbs = {  
    "give": {  
        "frame": "Verb + Object + Recipient",  
        "example": "Give the book to John",  
        "arguments": ["book", "John"]  
    },  
    "sleep": {  
        "frame": "Verb only",  
        "example": "Sleep",  
        "arguments": []  
    },  
    "put": {  
        "frame": "Verb + Object + Location",  
        "example": "Put the file on the desk",  
        "arguments": ["file", "desk"]  
    }  
}  
  
print("\nVERB: GIVE")  
  
print("Example:", verbs["give"]["example"])  
  
print("Frame:", verbs["give"]["frame"])  
  
print("Object:", verbs["give"]["arguments"][0])  
  
print("Recipient:", verbs["give"]["arguments"][1])  
  
print("\nVERB: SLEEP")  
  
print("Example:", verbs["sleep"]["example"])
```

```
print("Frame:", verbs["sleep"]["frame"])
```

```
print("Arguments: None")
```

```
print("\nVERB: PUT")
```

```
print("Example:", verbs["put"]["example"])
```

```
print("Frame:", verbs["put"]["frame"])
```

```
print("Object:", verbs["put"]["arguments"][0])
```

```
print("Location:", verbs["put"]["arguments"][1])
```

```
print("\nIMPORTANCE OF SUBCATEGORIZATION")
```

```
print("1. Identifies required verb arguments.")
```

```
print("2. Determines valid sentence structures.")
```

```
print("3. Helps detect missing arguments.")
```

```
print("4. Helps identify incorrect arguments.")
```

```
print("5. Supports semantic role labeling.")
```

```
print("6. Improves syntactic parsing.")
```

```
print("7. Helps conversational AI understand commands.")
```

```
print("\nEXAMPLE VALIDATION")
```

```
sentence1 = ["give", "book", "John"]
```

```
sentence2 = ["sleep"]
```

```
sentence3 = ["put", "file", "desk"]
```

```
if len(sentence1) == 3:
```

```
    print("Give -> Valid argument structure")
```

```
if len(sentence2) == 1:
```

```
    print("Sleep -> Valid argument structure")
```

```
if len(sentence3) == 3:
```

```
    print("Put -> Valid argument structure")
```

```
print("\nCONCLUSION")
```

```
print("Subcategorization frames specify the arguments required by verbs.")
```

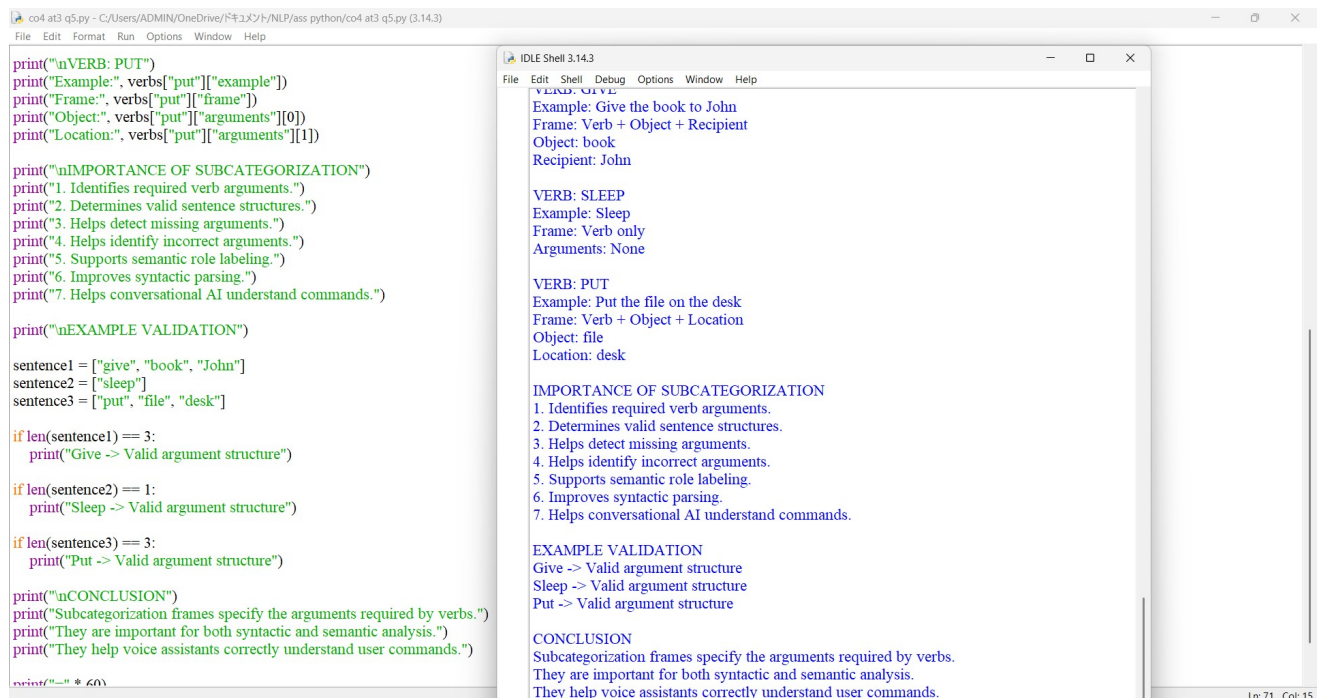
```
print("They are important for both syntactic and semantic analysis.")
```

```
print("They help voice assistants correctly understand user commands.")
```

```
print("=" * 60)
```

```
print("ANALYSIS COMPLETED")
```

```
print("=" * 60)
```



The image shows a screenshot of a Python script being executed in an IDLE Shell. The script defines three sentences and checks their lengths to determine if they have valid argument structures. It also prints a conclusion about subcategorization frames.

```
print("\nVERB: PUT")
print("Example:", verbs["put"]["example"])
print("Frame:", verbs["put"]["frame"])
print("Object:", verbs["put"]["arguments"][0])
print("Location:", verbs["put"]["arguments"][1])

print("\nIMPORTANCE OF SUBCATEGORIZATION")
print("1. Identifies required verb arguments.")
print("2. Determines valid sentence structures.")
print("3. Helps detect missing arguments.")
print("4. Helps identify incorrect arguments.")
print("5. Supports semantic role labeling.")
print("6. Improves syntactic parsing.")
print("7. Helps conversational AI understand commands.")

print("\nEXAMPLE VALIDATION")

sentence1 = ["give", "book", "John"]
sentence2 = ["sleep"]
sentence3 = ["put", "file", "desk"]

if len(sentence1) == 3:
    print("Give -> Valid argument structure")

if len(sentence2) == 1:
    print("Sleep -> Valid argument structure")

if len(sentence3) == 3:
    print("Put -> Valid argument structure")

print("\nCONCLUSION")
print("Subcategorization frames specify the arguments required by verbs.")
print("They are important for both syntactic and semantic analysis.")
print("They help voice assistants correctly understand user commands.")

print("=" * 60)
```

The output of the script is displayed in the IDLE Shell, showing the results of the argument structure checks and the conclusion.

```
VERB: PUT
Example: Give the book to John
Frame: Verb + Object + Recipient
Object: book
Recipient: John

VERB: SLEEP
Example: Sleep
Frame: Verb only
Arguments: None

VERB: PUT
Example: Put the file on the desk
Frame: Verb + Object + Location
Object: file
Location: desk

IMPORTANCE OF SUBCATEGORIZATION
1. Identifies required verb arguments.
2. Determines valid sentence structures.
3. Helps detect missing arguments.
4. Helps identify incorrect arguments.
5. Supports semantic role labeling.
6. Improves syntactic parsing.
7. Helps conversational AI understand commands.

EXAMPLE VALIDATION
Give -> Valid argument structure
Sleep -> Valid argument structure
Put -> Valid argument structure

CONCLUSION
Subcategorization frames specify the arguments required by verbs.
They are important for both syntactic and semantic analysis.
They help voice assistants correctly understand user commands.
```

Ln: 71 Col: 15

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1	4	4	4	4	4	20	92
2	4	4	3	3	4	18	
3	4	4	3	3	4	18	
4	4	4	3	3	4	18	
5	3	4	3	3	4	18	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____